Assignment 12

Bharat Unnati AI Fellowship

CodeXpert

Assignment 12

Name: Rohith Reddy B

USN: 1AY22AI076

College: ACHARYA INSTITUTE OF TECHNOLOGY

Email: rockrohith9683@gmail.com

Topic:

Core Logic: The Automobile Blueprint

Overview

Eachimplementation includes:

Properties:`brand`, `model`, `batteryLevel`

Logic:`calculateRange()` → *BatteryLevel×5kmper unit)*

Action:`displayStatus()` → Displays vehicle details

1. Java (Strict Object-Oriented)

```
class Automobile {   private String
brand;         private int batteryLevel;
       public Automobile(String b, int
bat) { this.brand = b;
this.batteryLevel = bat;
        }

                public int calculateRange() { return
                batteryLevel * 5;
        }

              public void displayStatus() {
 System.out.println("Brand: " + brand + " | Range:"+ calculateRange() + "km");
         }

               public static void main(String[] args) {
                Automobile myCar = new Automobile("Tesla", 80);
                myCar.displayStatus();
        }
}
```

2. Python (Dynamic & Concise)

```
class Automobile:   def __init__(self, brand,
battery):      self.brand = brand       self.battery =
battery

         def calculate_range(self):
                return self.battery * 5

       def display(self):         print(f"EV Brand: {self.brand}, Est. Range:
{self.cal culate_range()}km") my_car =
Automobile("Rivian", 90) my_car.display()
```

3. C++ (System Level)

```
#include <iostream> #include
<string> using namespace std;

class Automobile { public:
        string brand;   int
battery;

                    Automobile(string b, int bat) : brand(b), battery(bat) {}

  void display() {   cout << "Brand: " << brand << " | Range: " << battery  * 5 <<
"km" << endl;
          }
};

int main() {
        Automobile myCar("Lucid", 100);
        myCar.display();          return 0;
}
```

4. C# (.NET Enterprise)

```
using System; class Automobile {   public
string Brand { get;
set; }            public int Battery { get; set; }

                public Automobile(string b, int bat) {
                  Brand = b;
                  Battery = bat;
          }

              public void Show() {
 Console.WriteLine($"Vehicle: {Brand}, Range: {Battery * 5}km");
         }
```

```
}

classProgram { static void
        Main() {
                Automobile ev = new Automobile("Tesla", 75); ev.Show();
        }
}
```

5. JavaScript (Modern Web)

```
class   Automobile {
      constructor(brand, battery) {
            this.brand = brand;
            this.battery = battery;
      }

      display() {
            console.log(`EV: ${this.brand}, Range: ${this.battery
 * 5}km`);
      }
```

```
}

const myEV = new Automobile("Hyundai", 60);
myEV.display();
```

---

6. PHP (Server-Side Scripting)

```
<?php class
Automobile {
public $brand;  public
$battery;
            function __construct($b, $bat) {
```

```
        $this->brand = $b;
        $this->battery = $bat;
    }

  function show() {   echo "Brand: " . $this->brand . " | Range: " . ($this
->battery * 5) . "km";
     }
}

$car = new Automobile("Nio", 85);
$car->show();
?>
```

7. Ruby (Human-Centric)

```
class Automobile  def
initialize(brand, battery)
        @brand = brand
@battery = battery end  def display   puts "Brand: #{@brand}, Range:

#{@battery * 5}km"  end end my_car = Automobile.new("Audi e-tron",

70) my_car.display
```

8. Kotlin (Modern JVM)

```
class Automobile(val brand: String, val battery: Int) {       fun display() {
    println("Vehicle: $brand | Est. Range: ${battery * 5} km")
     }
}
fun main() {          val myCar = Automobile("Porsche", 95)
myCar.display()
}
```

Comparative Summary

Initialization:

Python uses `__init__`, Ruby uses `initialize`, while Java, C , and C# use constructors with the class name.

Variable Scope:

C and C# require explicit access modifiers (`public`, `private`), whereas Python and JavaScript are more flexible.

Output Formatting:

Modern languages like Python and Kotlin provide cleaner string formatting compared to traditional approaches.